Introduction à la programmation sous Android

Christophe Renaud

M2 Informatique

Année 2025-2026

Version 6.6.0 du 10/09/2025

Objectifs du cours

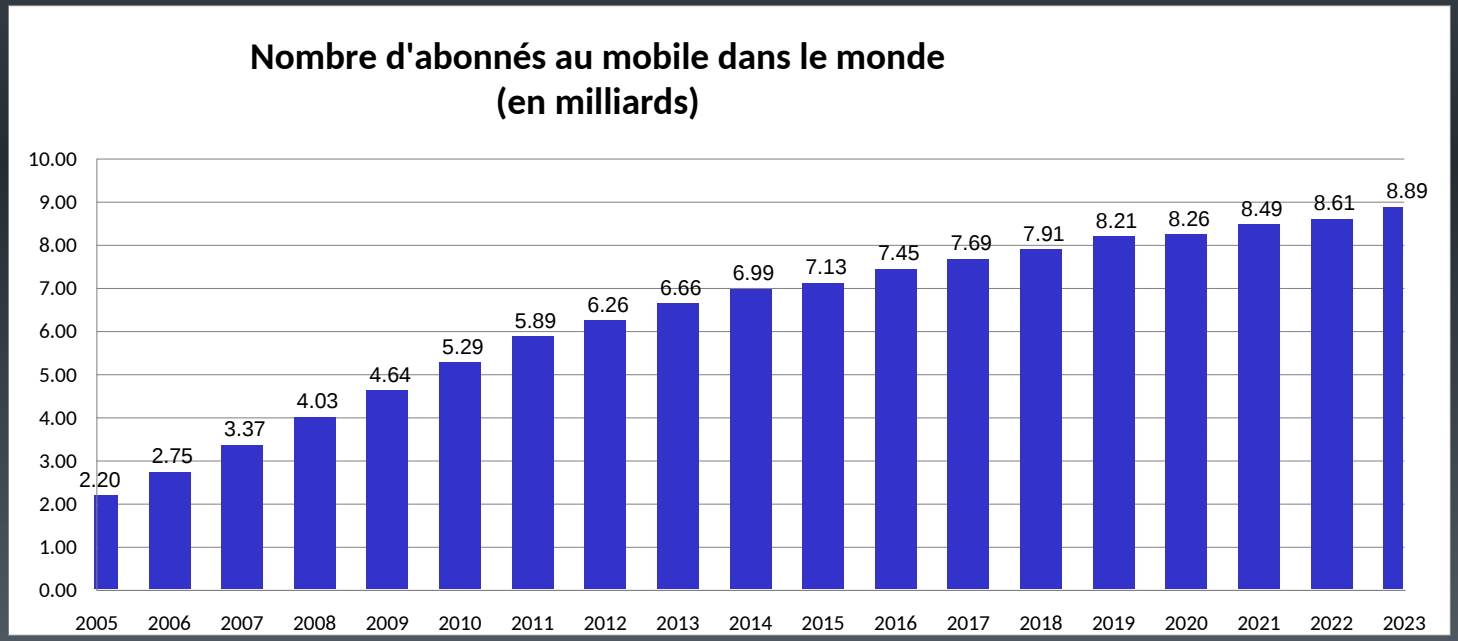
- Connaître les bases de la programmation sous Android
  - Environnement de développement (Android Studio)
  - Architecture d'une application
  - Modèle d'exécution

Plan du cours

- Introduction
- Architecture d'une application Android
- Les activités
- Définir une interface graphique
- Les intentions explicites
- Les intentions implicites
- Les menus
- Les listes
- Les permissions
- Les content providers

Développement mobile (1)

- Pourquoi développer des applications mobiles ?



https://www.itu.int/en/ITU-D/Statistics/Pages/stat/default.aspx

Développement mobile (2)

- Quelles solutions pour le développement ?

Natif

- IOS (objective C / swift)
- Android (java, kotlin)

+ bien adapté à l'OS ciblé
+ ergonomie/design optimal
+ notification push
+ autonomie réseau
- coût de développement (multiplateforme)

Hybride

- React Native (javascript)
- Ionic react (js, html, css)

+ Temps de développement réduit
+ Maintenance facilitée
- Performances moins stables

Web

- Vue.js

+ rien à installer
+ dvt rapide et portable
- fonctionnement sous-optimal
- besoin d'une connexion internet
- pas de notifications push

Flutter (langage Dart)

Android (2)

- Système d'exploitation à destination des dispositifs mobiles
  - Téléphones, tablettes, téléviseurs, montres, voitures, objets connectés
- Caractéristiques :
  - Opensource (licence Apache), gratuit, flexible
  - Basé sur un noyau linux
  - Inclut les applications de base (téléphone, sms, carnet d'adresse, navigateur, etc.)
  - Un ensemble important d'API (OpenGL, media, etc …)
  - Un SDK basé sur un sous-ensemble de JAVA (autres langages disponibles : Kotlin, C, C++, …)
  - Une machine virtuelle (Dalvik) qui exécute la majorité des applications
    - Remplacée par ART depuis la version 5.0 d'Android

Android (3)

- Historique :
  - Créé en 2005 par la société Android
  - Rachat en 2007 par Google
  - 21 versions depuis la 1.0 (Apple Pie) en 2008 jusqu'à la 14.0 en 10/2023.
  - Une version = 1 API
    - Apparition de nouvelles fonctionnalités
    - Modification de fonctionnalités existantes
    - Disparition de certaines fonctionnalités

version	Nom de code	API
4.0	Ice Cream Sandwich	15
4.1	Jelly Bean	16
4.2	Jelly Bean	17
4.3	Jelly Bean	18
4.4	Kitkat	19
5.0	Lollipop	21
5.1	Lollipop	22
6.0	Marshmallow	23
7.0	Nougat	24
7.1	Nougat	25
8.0	Oreo	26
8.1	Oreo	27
9.0	Pie	28
10.0	Q (Quince Tart)	29
11.0	R (Red Velvet Cake)	30
12.0	S (Snow Cone)	31-32
13.0	T (Tiramisu)	33
14.0	U(Upside Down Cake)	34
15.0	V(Vanilla Ice Cream)	35
16.0	B(aklava)	36

Android (1)

- Pourquoi développer sous Android ?

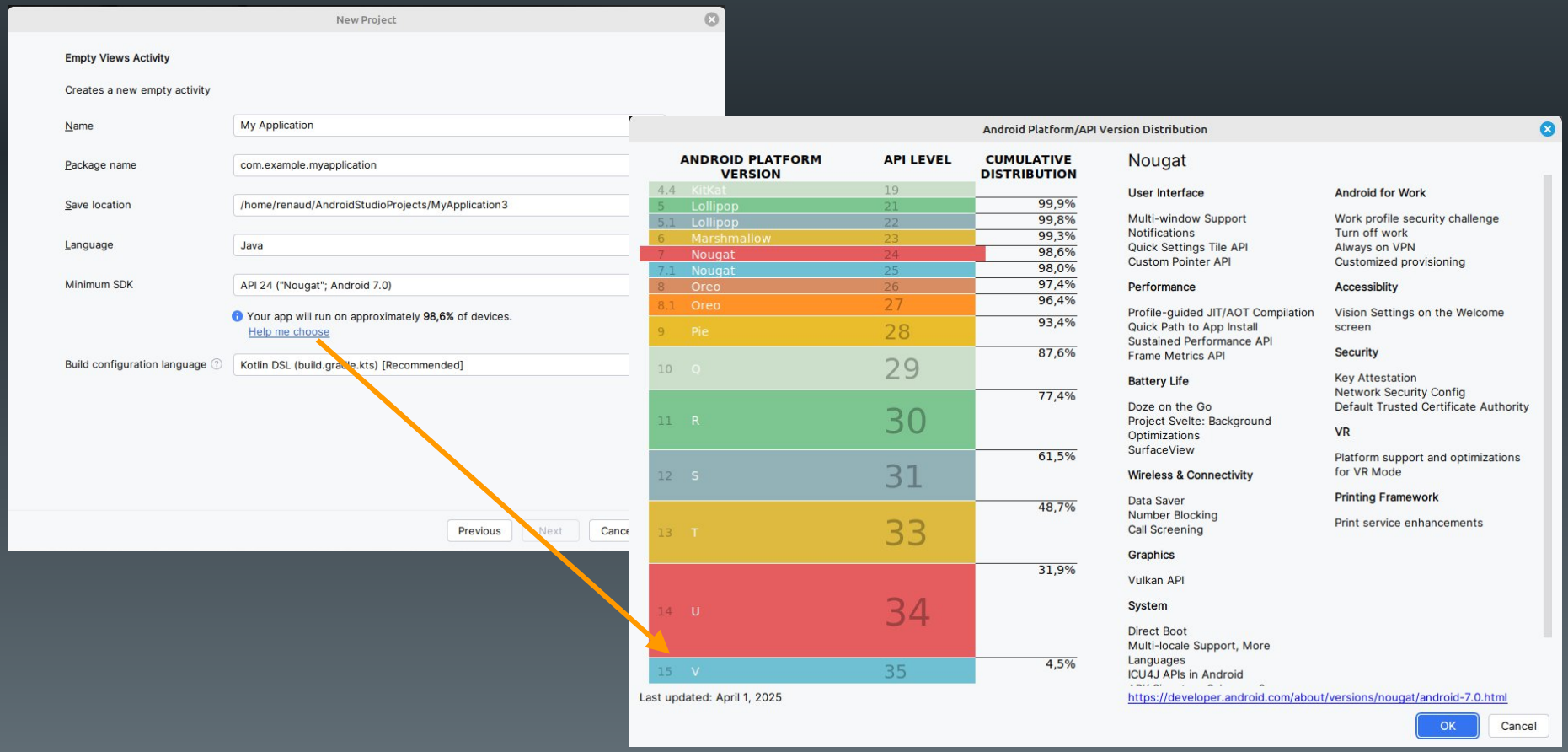
Répartition des ventes de smartphones selon leur OS (en %)

	2019	2020	2021	2022	2023	2024	2025
android	75,47	73,1	71,89	71,48	70,31	71,71	72,63
iOS	22,70	26,3	27,34	27,84	28,97	27,67	26,97
Autres	1,83	0,6	0,77	0,68	0,72	0,62	0,40

Source (Août 2025) : https://gs.statcounter.com/os-market-share/mobile/worldwide

Android (4)

- Comment assurer la compatibilité ?
  - Vérifier depuis Android Studio lors de la création d'un nouveau projet



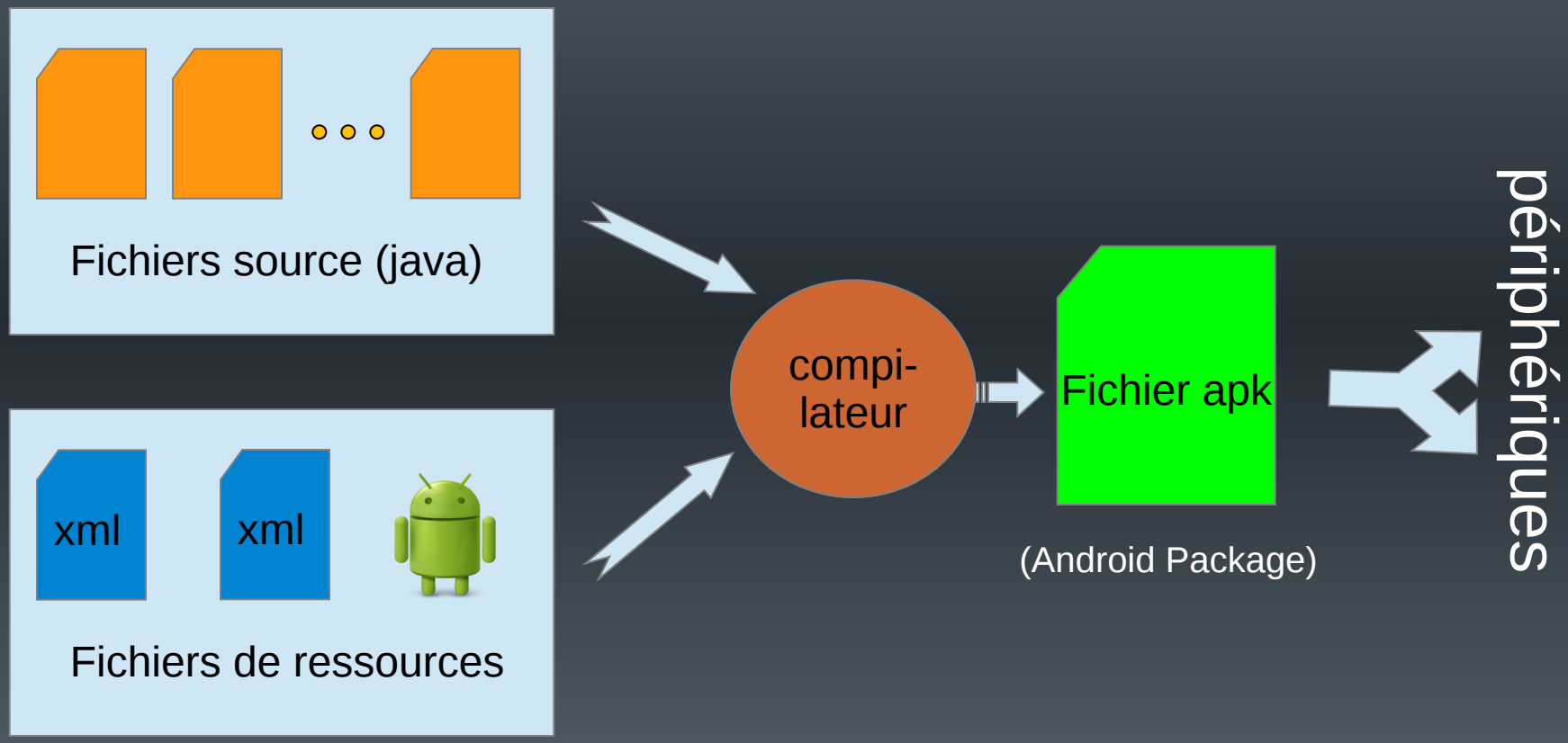
Android (5)

- Les contraintes
  - Hétérogénéité du matériel
    - Processeurs, mémoire
    - Écrans
    - Dispositifs spécialisés
  - Puissance et mémoire limitées
  - Interface tactile
  - Connectivité à internet (disponibilité, rapidité, ...)
  - Développement extérieur au périphérique

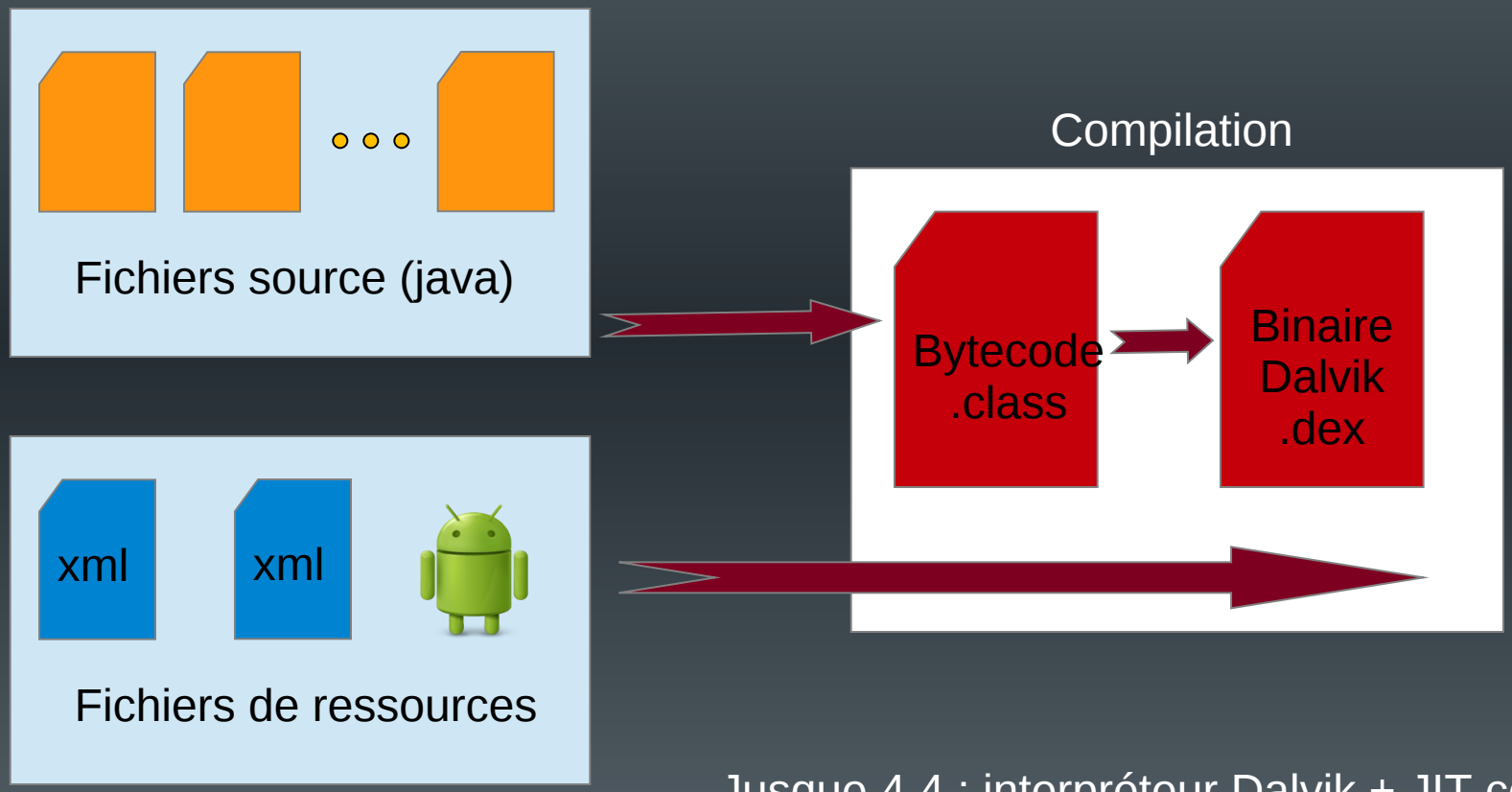
Plan du cours

- Introduction
- Architecture d'une application Android
- Les activités
- Définir une interface graphique
- Les intentions explicites
- Les intention implicites
- Les permissions
- Les menus
- Les listes
- Les content providers

Schéma de développement



La compilation

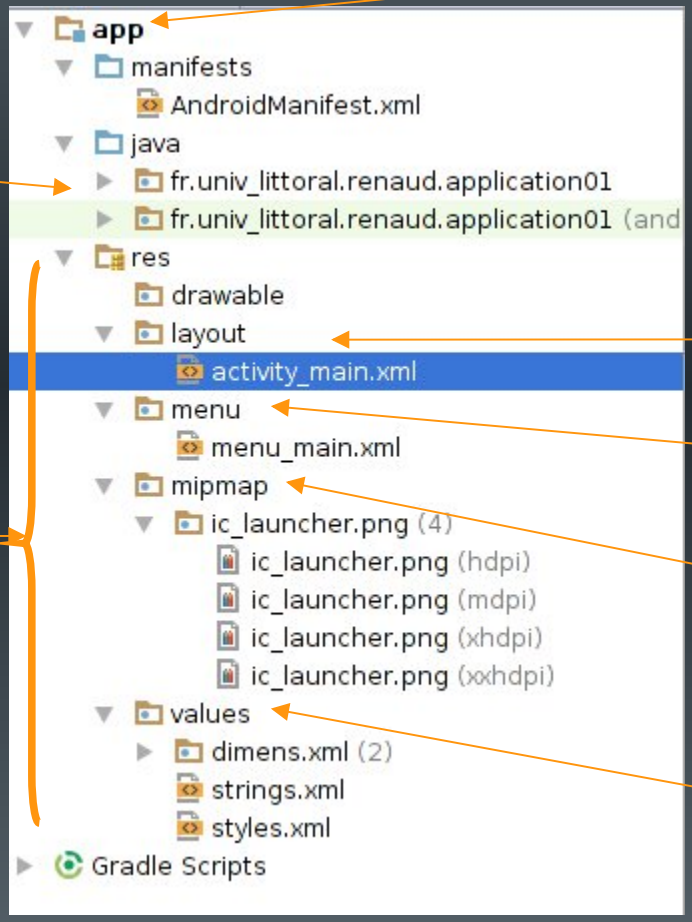


Jusque 4.4 : interpréteur Dalvik + JIT compilation de parties « critiques »

À partir de 5.0 : ART (Android RunTime)

- compilation en code natif sur le support

Architecture d'un projet



Dossier du projet

Sources Java

Mises en page

Menus

Les ressources

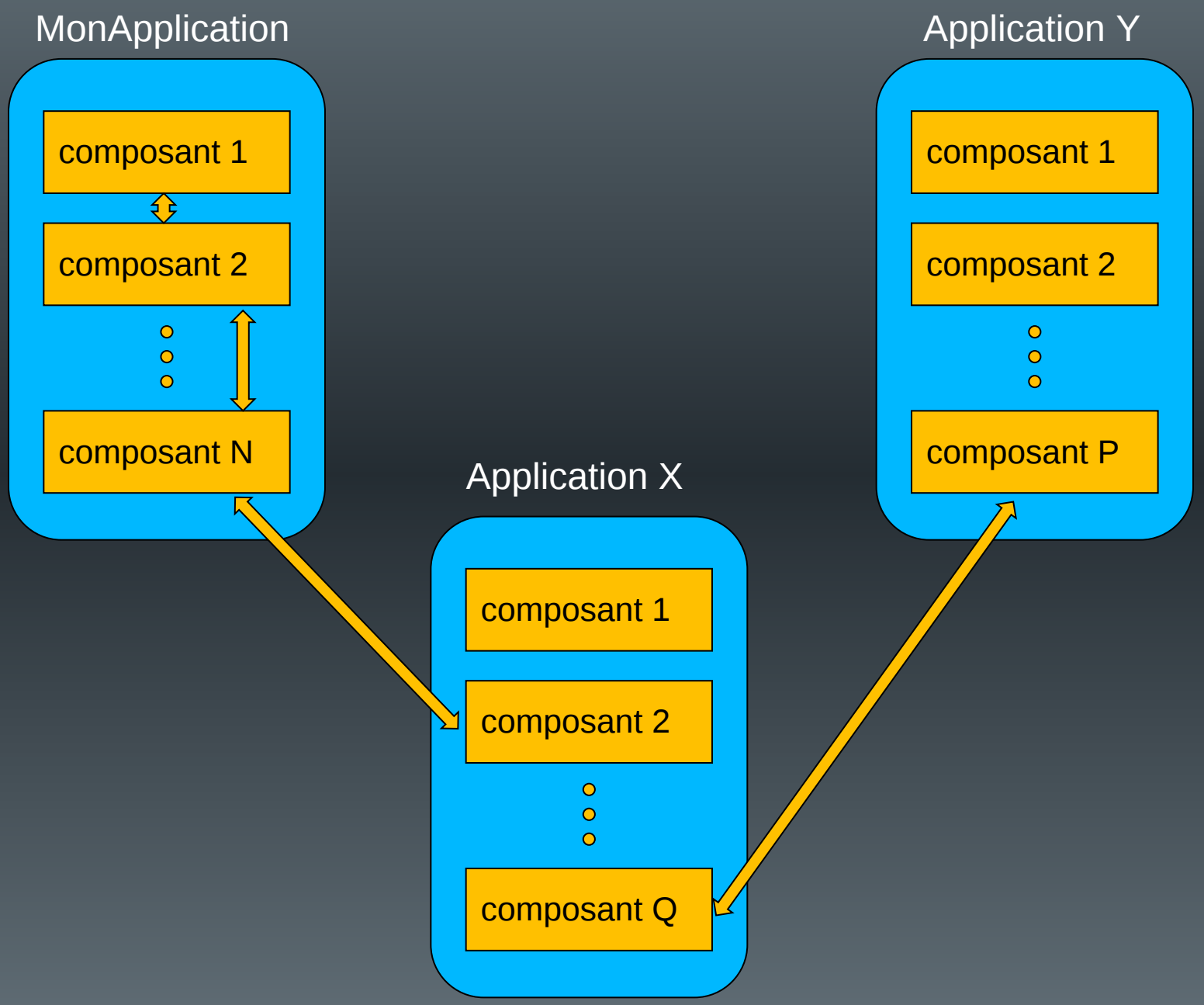
Icônes

Données « constantes »

(sous Android Studio)

Les éléments d'une application

- Une application = {composants}
- Les composants :
  - Existent de manière indépendante
  - Vus comme autant de points d'entrée par le système
    - Pas de « main » dans une application
- Liés au design d'Android :
  - Toute application doit pouvoir démarrer un composant d'une autre application (sous réserve de droits) et récupérer ses « résultats »


MonApplication
composant 1
composant 2
composant N
Application Y
composant 1
composant 2
composant P
Application X
composant 1
composant 2
composant Q

Exemple

- Mon application = application d’effets sur un portrait de l'utilisateur

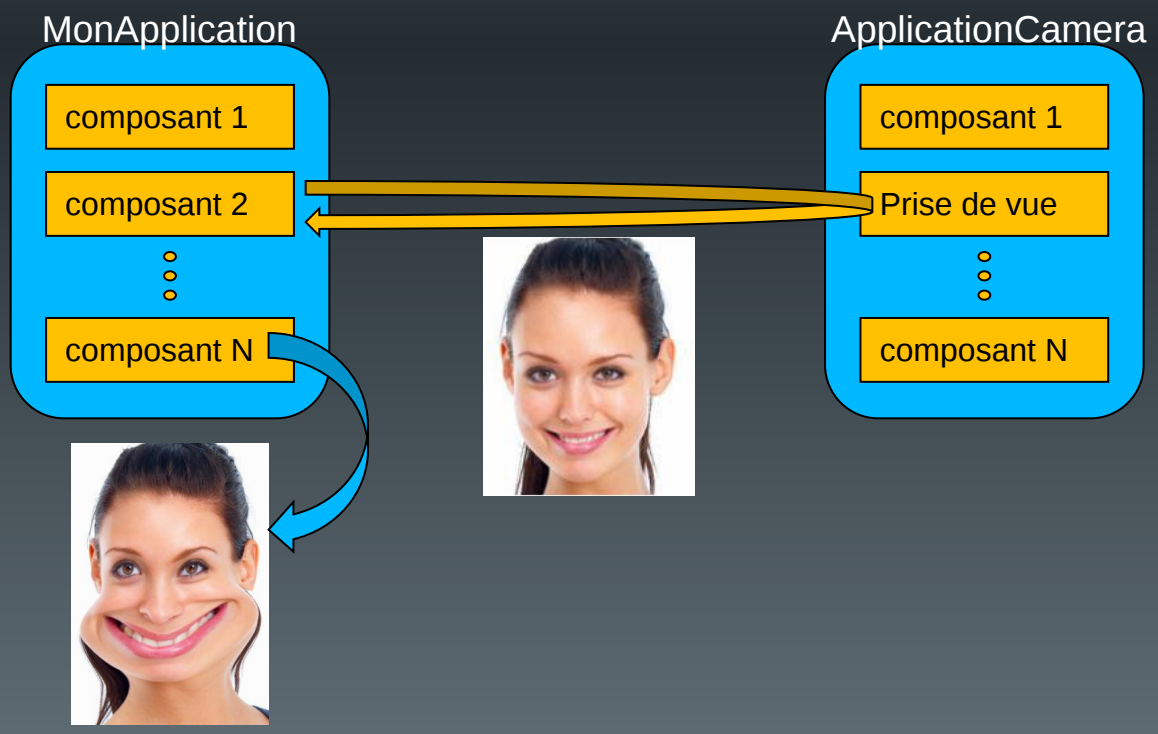
Ma spécialité : traitement d’images

La difficulté : écrire le code de gestion de l'appareil photo embarqué

Source : google play - effets du visage

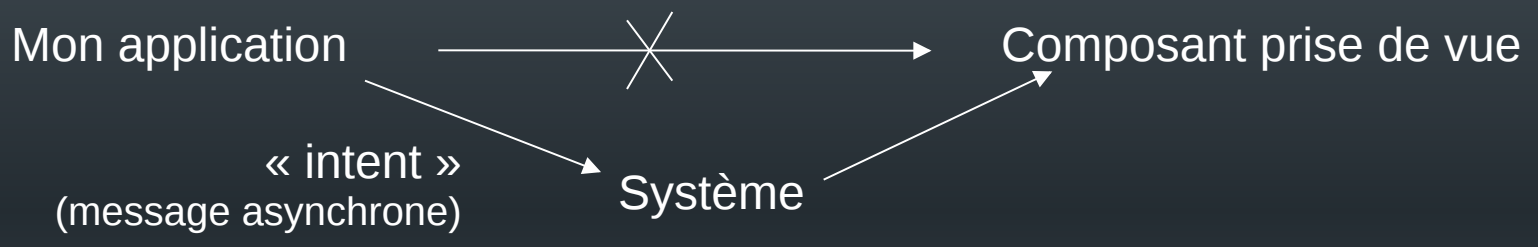
Exemple

- Android :
  - démarrage d'un composant existant permettant la prise de vue
  - Récupération de l'image



Remarques

- Problèmes de droits :



- Problèmes d'information :
  - Le système doit connaître le rôle particulier de certains composants
    - Ce sont les applications qui enregistrent ces informations auprès du système (cf filtres d'intention)

Les composants

- Les activités (Activity)
  - Un écran avec une interface utilisateur et un contexte
- Les services (Service)
  - Composant sans interface, qui tourne en fond de tâche (lecteur de musique, téléchargement, ...)
- Les fournisseurs de contenu (ContentProvider)
  - I/O sur des données gérées par le système ou par une autre application
- Des récepteurs d'intentions (BroadcastReceiver)
  - Récupération d'informations générales
    - arrivée d'un sms, batterie faible, ...

Les interactions

- Les intentions (*Intent*)
  - Permet d’échanger des informations entre composants
    - Démarrage d’un composant en lui envoyant des données
    - Récupération de résultats depuis un composant
    - Recherche d’un composant en fonction d’un type d’action à réaliser
- Les filtres d'intentions (<intent-filter>)
  - Permet à un composant d'indiquer ce qu'il sait faire
  - Permet au système de sélectionner les composants susceptibles de répondre à une demande de savoir-faire d’une application

AndroidManifest.xml

- Description de l'application
  - Liste des composants
  - Niveau minimum de l'API requise
  - Liste des caractéristiques physiques nécessaires
    - Évite d'installer l'application sur du matériel non compatible (gestion de la visibilité sur Google Play)
  - Liste des permissions dont l'application a besoin
  - Liste des autres API nécessaires
    - ex. Google Map
  - Etc.
- <u>Généré automatiquement par Android Studio</u>

Exemple

```
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
      package="fr.univ_littoral.renaud.bidon" >

<application
        android:allowBackup="true"
                android:icon="@mipmap/ic_launcher"
                android:label="@string/app_name"
                android:theme="@style/AppTheme" >
                <activity
                android:name=".MainActivity"
                android:label="@string/app_name" >
                <intent-filter>
                        <action android:name="android.intent.action.MAIN" />
                        <category android:name="android.intent.category.LAUNCHER" />
                </intent-filter>
                </activity>
 </application>
</manifest>
```

Les ressources

- Ressources = toutes les données (autres que le code) utilisées par l'application
- Rangées dans le dossier **res**, puis incluses dans l'apk
  - res/drawable et res/mipmap (images en différentes résolutions)
  - Layout (description en XML des interfaces)
  - Menus (description en XML des menus)
  - Values (définitions en XML des constantes utilisées par l'application : chaînes, tableaux, valeurs numériques, etc.)

strings.xml

- Fichier ressources, contenant toutes les chaînes constantes
  - Principalement utilisées pour l'interface

```
<resources>
    <string name="app_name">MyApplication</string>

    <string name="hello_world">Hello world!</string>
    <string name="action_settings">Settings</string>
</resources>
```

Type de la constante

Nom de la constante (permet l'appel depuis l'application ou un autre fichier XML)

Valeur de la constante

Internationalisation

- Objectif :
  - Disposer de plusieurs versions des textes, libellés, etc utilisés par l'application
  - Choix automatique des textes en fonction de la configuration du périphérique
- Principe
  - Dupliquer le fichier strings.xml : 1 version par langue supportée
  - Stocker chaque version dans un dossier spécifique
    - values-xx (ex. values-en, values-fr, …)
  - Géré via Android Studio

```
app/
  res/
    values/
        strings.xml
    values-en/
        strings.xml
    values-fr/
        strings.xml
```

La classe R

- Classe générée par l'IDE
  - Permet l'accès aux ressources
  - Créée à partir de l'arborescence présente dans le dossier **res**
  - Elle contient des classes internes dont les noms correspondent aux différents types de ressources (drawable, layout, …)
  - Elle contient des propriétés permettant de représenter l'ensemble des ressources de l'application
- Utilisation en Java :
  - R. type.identificateur

```
<resources>
   <string name="app_name">MyApplication</string>

   <string name="hello_world">Hello world!</string>
   <string name="action_settings">Settings</string>
</resources>
```

R.string.app_name

R.string.hello_world

R.string.action_settings

Référencement des ressources en XML

- Forme générale : @type/identificateur

```
<resources>
   <string name="app_name">MyApplication</string>

   <string name="hello_world">Hello world!</string>
   <string name="action_settings">Settings</string>
</resources>
```

@string/app_name

@string/hello_world

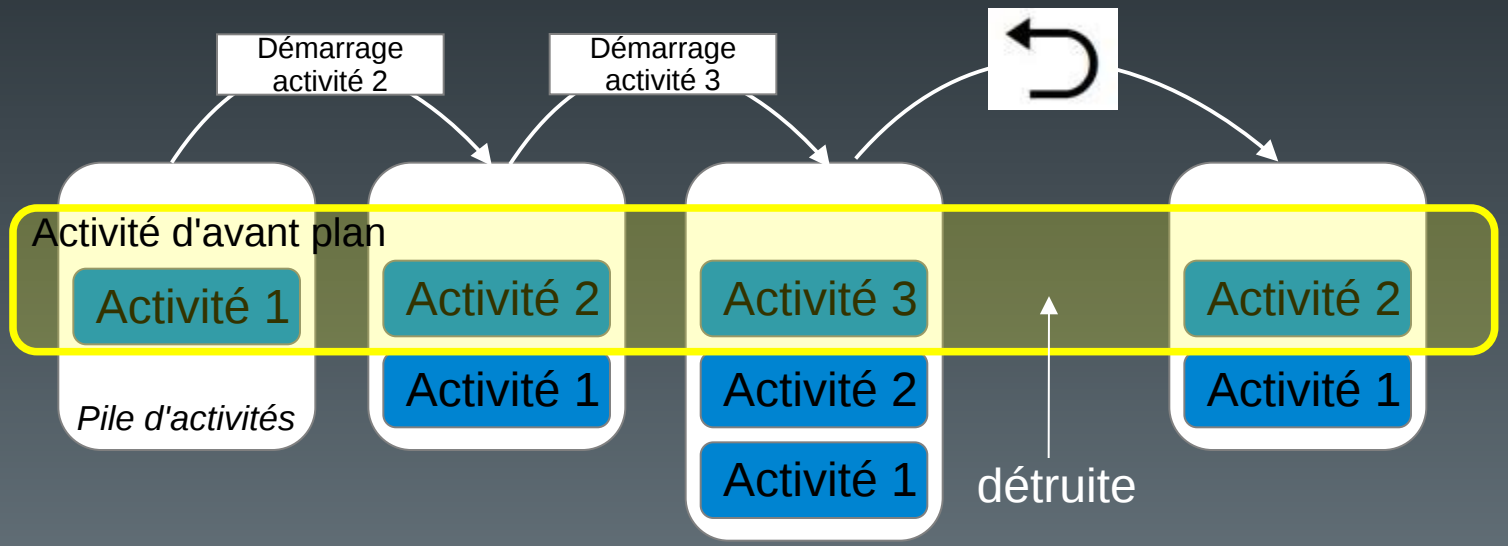
@string/action_settings

Plan du cours

- Introduction
- Architecture d'une application Android
- Les activités
- Définir une interface graphique
- Les intentions explicites
- Les intentions implicites
- Les menus
- Les listes
- Les permissions
- Les content providers

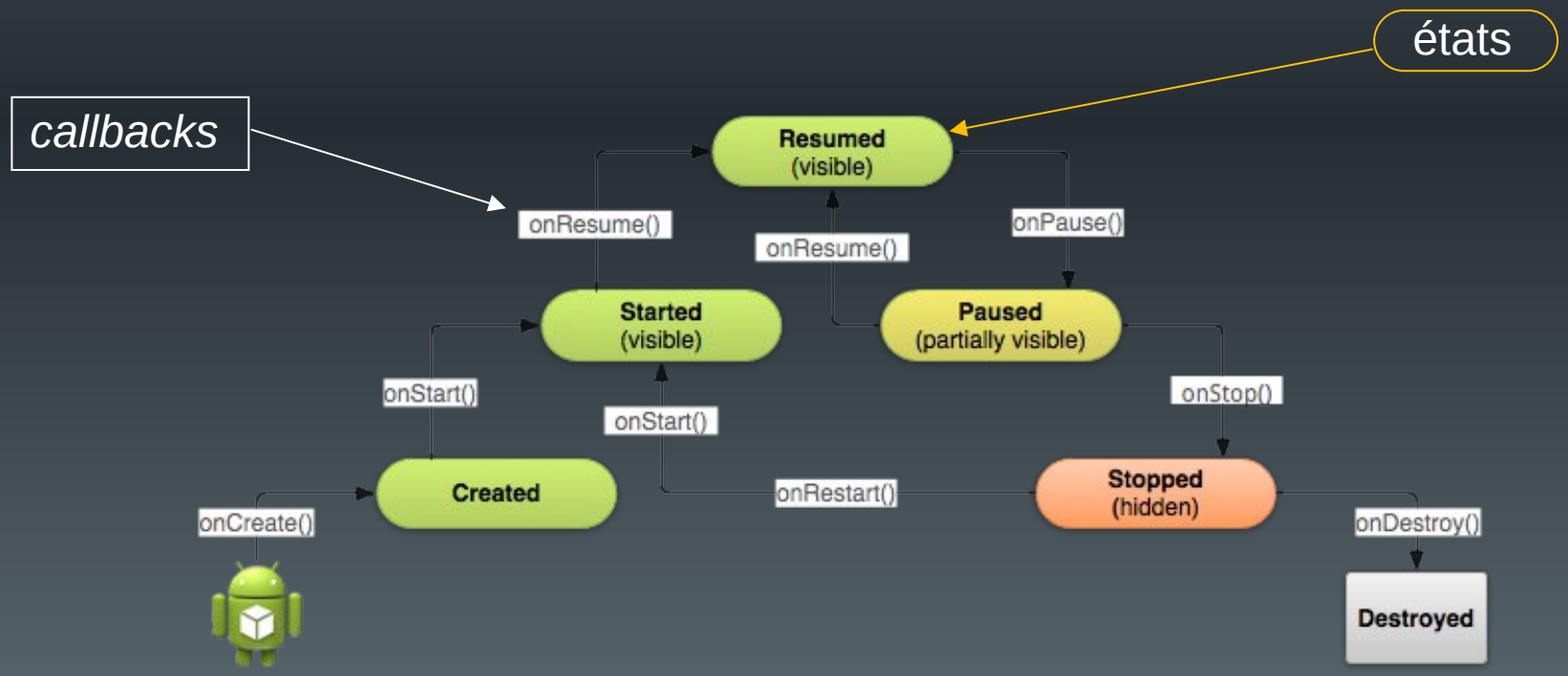
Les activités (1)

- Un composant d'une application, doté d'une interface graphique (IHM) et d'un contexte
- Une activité à la fois visible de l'utilisateur
  - Pour une même application
  - Pour des applications différentes
- Empilement des activités



Les activités (2)

- Cycle de vie
  - Une activité peut se trouver dans différents états en fonction des actions du système et/ou de l'utilisateur :



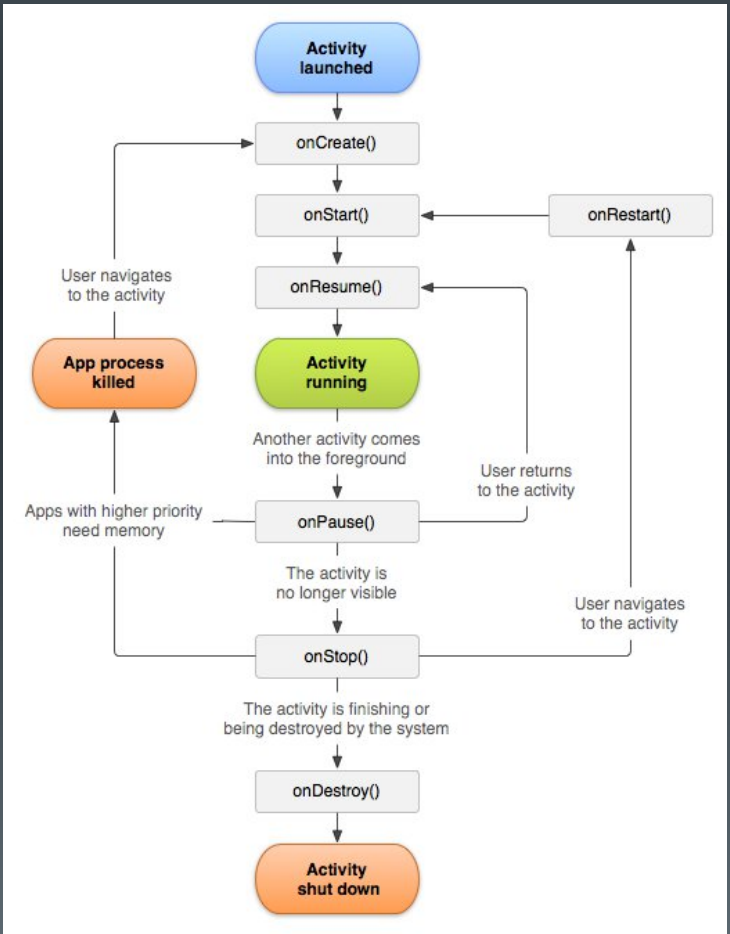
Les activités (3)

- Développement
  - Une classe java par activité ;
  - Les ressources associées (layout, menu, etc.) ;
  - La classe hérite de la classe AppCompatActivity ;
  - Génération d'un code minimum par défaut sous Android Studio.

```
…

public class Bidon extends AppCompatActivity {

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.Bidon);

            // initialisation des variables internes
    …
    }

    …
}
```

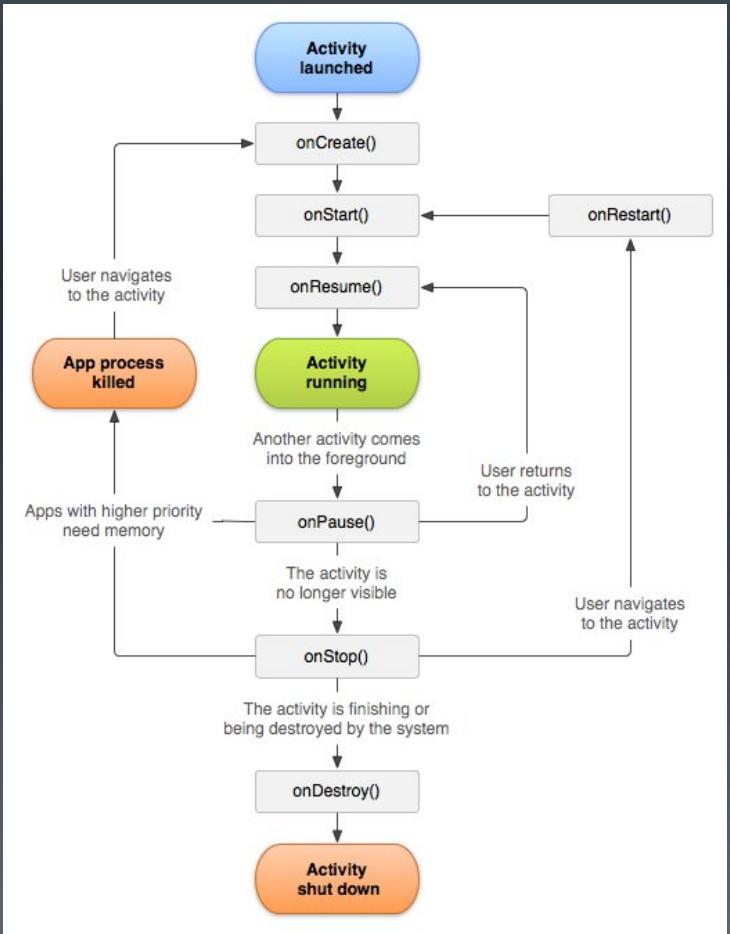
Les activités (4)

- D'autres méthodes peuvent être surchargées, en précisant ce qui doit être fait quand :
  - protected void onDestroy()
    - L'activité se termine
    - Libérer toutes les ressources utilisées
  - protected void onStop()
    - L'activité n'est plus visible
    - Stopper les ressources qui ne sont plus visibles (ex. animations)
  - protected void onPause()
    - L'activité n'est plus au premier plan mais est encore visible (superposition d'une fenêtre de dialogue, multifenêtrage)
    - Stopper les ressources non utilisées (ex. capteur GPS, camera, …)



Les activités (4)

- D'autres méthodes peuvent être surchargées, en précisant ce qui doit être fait quand :
  - protected void onStart()
    - l'activité devient visible
    - Redémarrer les ressources stoppées dans onStop()
  - protected void onResume()
    - l'activité est au premier plan
    - Redémarrer les ressources stoppées dans onPause()
  - protected void onRestart()
    - l'activité redevient visible



Les activités (5)

- Destruction de l'application par le système
  - Cas normal : l'activité est terminée. Le système récupère les ressources, en particulier la mémoire
  - Cas spéciaux : suppression d'une activité non active pour des raisons :
    - de limites des ressources ;
    - de changement d'orientation de l'écran ou de multifenêtrage
  - Le système doit sauvegarder l'état de l'activité, pour pouvoir la redémarrer dans son état courant
    - Sauvegarde dans un objet **Bundle** : couples (nom_donnée, valeur)
      - Contient les données utilisées par l'interface par défaut
      - Systématique dès que l'activité n'est plus visible
    - Surcharge des méthodes de sauvegarde et restauration si d'autres données doivent être sauvées

Les activités (6)

outState

("playerName" , "toto")
("playerPower" , 123.890)
("autreClé01" , autreValeur01)
("autreClé02" , autreValeur02)
…

- Sauvegardes
  - **void onSaveInstanceState(Bundle outState)**

```
public final static String PLAYER_NAME_KEY = "playerName"
public final static String PLAYER_POWER_KEY = "playerPower"
```

Constantes définissant le nom des clés

```
String nomJoueur;
float  puissance;
```

Variables internes

```
// Fonction callback appelée en cas de destruction temporaire de l'activité
@Override
public void onSaveInstanceState(Bundle outState) {

    outState.putString(PLAYER_NAME_KEY, nomJoueur);
    outState.putFloat(PLAYER_POWER_KEY, puissance);

    // appel à la super classe pour sauvegarder les données de l'interface
    super.onSaveInstanceState(outState);
}
```

Sauvegarde de la valeur des variables internes dans le bundle

Les activités (7)

- Récupération
  - **void onCreate(Bundle savedInstanceState)**

```
@Override
public void onCreate(Bundle savedInstanceState) {
   // appel à la super classe pour mettre à jour les données de l'interface
   super.onCreate(savedInstanceState);

   // récupération des données internes
   if (savedInstanceState != null) {
      nomJoueur = savedInstanceState.getString(PLAYER_NAME_KEY);
      puissance = savedInstanceState.getFloat(PLAYER_POWER_KEY);
}
```

  - **void onRestoreInstanceState (Bundle savedInstanceState)**

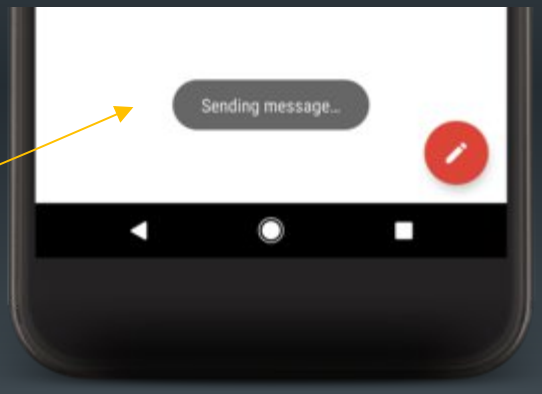
```
// Callback appelé après onStart() uniquement si un appel à onSaveInstanceState()
//  a été fait. Le bundle savedInstanceState est le même que pour onCreate()
@Override
public void onRestoreInstanceState(Bundle savedInstanceState) {
      nomJoueur = savedInstanceState.getString(PLAYER_NAME_KEY);
      puissance = savedInstanceState.getFloat(PLAYER_POWER_KEY);
}
```

TP 01

Remarques

https://www-lisic.univ-littoral.fr/~renaud/

- Affichage de messages de mises au point
  - Possibilité d'utiliser *System.out.println*
    - Affichage dans la console d'Android Studio
  - La classe Toast



Source : develop.android.com

Le message

Toast.*makeText*(**this**, **"Sending message…"**, Toast.***LENGTH_SHORT***).show();

Le contexte

La durée d'affichage

Le déclenchement de l'affichage